

DevOps Bootcamp Training Materials

Modules 1-5: Git, Docker, Kubernetes, Monitoring & Incident Response | Platform Team

1. Introduction to TechViet's DevOps Culture

Welcome to the TechViet DevOps Bootcamp. This training program is designed for newly hired engineers and engineers transitioning to DevOps responsibilities. Over the next two weeks, you will learn the tools, processes, and cultural practices that make up TechViet's DevOps approach. By the end of this bootcamp, you will be comfortable with: our deployment pipeline and how to deploy code to production, our monitoring and alerting infrastructure, our incident response process, our infrastructure-as-code practices, and the operational aspects of our Kubernetes platform. DevOps at TechViet is not a separate team or role — it is a set of practices shared across all engineering teams. While we do have a dedicated Platform Engineering team that builds and maintains the shared infrastructure, every engineer is expected to understand how their code is deployed, monitored, and operated in production. This 'you build it, you run it' philosophy means that the team that writes a service is also the team that carries the pager for that service. The Platform team provides the tools and abstractions that make this manageable, but the operational responsibility lies with each product team. This bootcamp is organized into hands-on modules, each with explanatory material followed by practical exercises that you will complete in the TechViet sandbox environment (a dedicated AWS account and Kubernetes cluster provisioned for training). Each module builds on the previous one, and by the end of Module 8, you will have deployed a complete application to the sandbox environment, set up monitoring and alerts, and practiced responding to a simulated incident.

2. Module 1: Git Workflow and CI Pipeline

TechViet uses a trunk-based development workflow with short-lived feature branches. All repositories are hosted on GitHub Enterprise (github.techviet.vn) and follow the same branching model: the 'main' branch represents the current production state and is protected — direct pushes are not allowed. All changes go through pull requests with the following requirements: at least 2 approving reviews (enforced by GitHub branch protection), all CI checks passing (enforced by required status checks), no merge conflicts with the main branch, and a linked Jira ticket for traceability. The CI pipeline is implemented using GitHub Actions and runs automatically on every push to a pull request branch. The pipeline consists of the following stages: Stage 1 — Lint and Format: runs language-specific linters (golangci-lint for Go, flake8 + black for Python, ESLint + Prettier for TypeScript, checkstyle for Java) and fails if any formatting or linting violations are detected. This stage typically completes in 30-60 seconds. Stage 2 — Unit Tests: runs the project's unit test suite with code coverage measurement. The coverage threshold varies by project but is typically 80% for new code (measured by diff coverage using the Codecov GitHub integration). This stage typically completes in 2-5 minutes depending on the test suite size. Stage 3 — Integration Tests: for services that depend on databases or external services, this stage spins up dependent services using Docker Compose (or Testcontainers for Java projects) and runs the integration test suite. Database-dependent tests use an isolated database instance provisioned for the test run, which is destroyed after the tests complete. This stage typically takes 5-15 minutes. Stage 4 — Security Scan: runs Semgrep for static application security testing (SAST) and Trivy for container image vulnerability scanning. Findings above the MEDIUM severity

threshold fail the pipeline. Exceptions for known false positives are documented in the .semgrep-ignore file in each repository. Stage 5 — Build and Push: if all previous stages pass, the pipeline builds the Docker image, tags it with the git commit SHA, and pushes it to our container registry (Amazon ECR). The image is also tagged with the branch name for easy identification. This stage takes 2-5 minutes depending on the image size and layer caching effectiveness. The total CI pipeline typically completes in 10-25 minutes. If any stage fails, the entire pipeline stops and the engineer is notified via GitHub comment and Slack notification. Engineers are expected to fix failing pipelines as a top priority — a red CI pipeline blocks the entire team from merging.

3. Module 2: Containerization and Docker Best Practices

All TechViet services are containerized using Docker. Understanding container concepts and best practices is essential for debugging production issues, optimizing build times, and ensuring security. Dockerfile best practices enforced at TechViet: use multi-stage builds to minimize image size. The first stage (builder) includes build tools and dependencies, while the final stage copies only the compiled binary or runtime dependencies. This typically reduces image size by 60-80%. Use specific base image tags (e.g., golang:1.22.1-bookworm) rather than generic tags (e.g., golang:latest) to ensure reproducible builds. Pin dependency versions in the package manager (go.sum, requirements.txt with hashes, package-lock.json). Run the application as a non-root user. All our Dockerfiles include a 'USER appuser' directive that creates a non-root user and runs the process as that user. Include a HEALTHCHECK instruction that tells Docker (and Kubernetes) how to check if the container is healthy. Order Dockerfile instructions from least-frequently changed to most-frequently changed to maximize layer caching. Typically: base image, system packages, dependency installation, source code copy, build command. Do not include secrets, credentials, or configuration in the Docker image. All runtime configuration is injected via environment variables or mounted configuration files. Image scanning: every Docker image is scanned by Trivy before being pushed to ECR and again on a weekly schedule (to catch newly discovered vulnerabilities in already-deployed images). The weekly scan results are reported to the Security team, who triage findings and create tickets for remediation. Critical and High severity vulnerabilities in the base OS packages must be patched within 7 days; application dependency vulnerabilities are prioritized based on exploitability.

4. Module 3: Kubernetes Fundamentals

This module covers the Kubernetes concepts that every TechViet engineer needs to understand for day-to-day work. We assume no prior Kubernetes experience. Kubernetes is a container orchestration platform that automates the deployment, scaling, and management of containerized applications. At TechViet, we run Amazon EKS (Elastic Kubernetes Service), which provides managed Kubernetes control plane nodes while we manage the worker nodes where our applications run. Key concepts you will use daily: Pod: the smallest deployable unit in Kubernetes, consisting of one or more containers that share network and storage. In practice, most TechViet pods run a single application container plus a sidecar container for the service mesh proxy (AWS App Mesh Envoy proxy). Pods are ephemeral — they can be killed, restarted, or rescheduled at any time. Never assume that a pod will run on the same node or retain local state across restarts. Design your application to be stateless (store state in the database or Redis, not in local memory or filesystem). Deployment: a Kubernetes resource that manages a set of identical pods and ensures that the desired number of replicas are running at all times. When you deploy a new version of your service, you update the Deployment's container image tag, and Kubernetes performs a rolling update: it gradually creates new pods with the new image and terminates old pods, maintaining the desired replica count throughout. Our deployments are configured with a maxSurge of 25% and maxUnavailable of 0%, meaning Kubernetes will create new pods before terminating old ones, ensuring zero-downtime deployments. Service: a Kubernetes resource that provides a stable network endpoint

(DNS name and IP address) for a set of pods. Even though pods come and go, the Service's address remains constant. TechViet uses ClusterIP Services for internal communication (accessible only within the cluster) and LoadBalancer Services for external-facing endpoints (which provision an AWS ALB). ConfigMap and Secret: Kubernetes resources for injecting configuration and sensitive data into pods as environment variables or mounted files. At TechViet, non-sensitive configuration (feature flags, service endpoints) is stored in ConfigMaps managed in the GitOps repository. Sensitive data (database passwords, API keys) is stored in AWS Secrets Manager and synced to Kubernetes Secrets using the External Secrets Operator, which ensures that secrets are never stored in git. Namespace: a virtual partition of the Kubernetes cluster that provides isolation between teams and environments. TechViet's production cluster has namespaces for: chatassist-prod (the ChatAssist microservices), portal-prod (the Customer Portal), platform-prod (shared platform services like monitoring and logging), and system (Kubernetes system components and operators).

5. Module 4: Monitoring and Observability

Observability is the ability to understand the internal state of a system by examining its external outputs. At TechViet, observability is built on three pillars: metrics (numerical measurements of system behavior over time), logs (timestamped records of discrete events), and traces (records of the path a request takes through the distributed system). Metrics: TechViet uses Datadog as the primary metrics platform. Every service emits metrics using the StatsD protocol (for application-level metrics) and Prometheus (for Kubernetes-level metrics). Key metrics that every engineer should know: request rate (requests per second, broken down by endpoint, status code, and method), error rate (percentage of requests returning 5xx status codes), latency distribution (p50, p95, p99 response times), saturation (CPU utilization, memory utilization, database connection pool utilization), and business metrics (specific to each service — e.g., messages per second for the Conversation Service, model inference latency for the AI Engine). Every service must define Service Level Objectives (SLOs) based on these metrics. An SLO is a target value for a metric that represents the minimum acceptable performance. For example, the Conversation Service's SLOs include: availability > 99.95% (measured as the percentage of successful HTTP responses in a rolling 30-day window), p99 message send latency < 200ms, and p99 WebSocket delivery latency < 300ms. SLOs are visualized on the team's Datadog dashboard and tracked as error budgets: if the SLO is 99.95% availability, the error budget is 0.05% — approximately 21 minutes of downtime per month. When a team exhausts more than 50% of their error budget in a month, they are expected to prioritize reliability work over feature development. Logs: all services emit structured JSON logs using the common log schema defined in the Engineering Standards. Logs are collected by the Datadog Agent running as a DaemonSet on every Kubernetes node, processed through the Datadog log pipeline (which applies parsing rules, enriches with Kubernetes metadata, and extracts key fields), and stored in Datadog Log Management with 30-day retention for standard logs and 90-day retention for security-related logs. Every log entry must include: timestamp, service name, environment, log level (DEBUG, INFO, WARN, ERROR), request_id (for correlation with traces), and a human-readable message. Traces: distributed tracing is implemented using OpenTelemetry, which propagates a trace ID across all services that handle a request. This allows engineers to follow a single user request from the load balancer, through the Conversation Service, to the AI Engine, to the database, and back — seeing the latency contribution of each component. Traces are exported to Datadog APM, where they can be visualized as flame graphs and correlated with logs and metrics. Every external-facing service instruments request handling with spans, and internal libraries for database queries, Redis operations, and HTTP client calls automatically create child spans.

6. Module 5: Incident Response

Every TechViet engineer participates in the on-call rotation for their team's services. This module prepares you for your first on-call shift by explaining the incident response process, the tools you will use, and common troubleshooting patterns.

On-call rotation: each product team has a primary and secondary on-call rotation, with weekly shifts starting on Monday at 09:00 Vietnam time. The primary on-call engineer is the first responder for all alerts from the team's services. The secondary on-call is the escalation point if the primary is unreachable or needs assistance. On-call engineers receive a monthly stipend of VND 3,000,000 and an additional VND 500,000 for each night or weekend hour spent actively responding to incidents.

Alert routing: alerts from Datadog flow through PagerDuty, which routes them to the appropriate on-call engineer based on the service and severity. Alert severities are:

- P1 (Critical — immediate page via phone call and SMS):** the service is down or severely degraded, affecting a large number of users. Expected response time: 5 minutes.
- P2 (High — page via push notification and SMS):** the service is degraded but still functional, or a small number of users are affected. Expected response time: 15 minutes.
- P3 (Medium — push notification only):** a non-urgent issue that should be investigated during business hours. Expected response time: next business day.
- P4 (Low — email only):** informational alert, no action required.

When you receive a P1 or P2 alert: acknowledge the alert in PagerDuty within the expected response time (this stops the escalation timer). Assess the situation by checking the service's Datadog dashboard and the most recent alerts in the #alerts Slack channel. If the issue is within your ability to resolve, proceed with diagnosis and mitigation. If you need help, escalate to the secondary on-call or post in the team's Slack channel. For P1 incidents, open the #incident-[number] Slack channel (using the /incident Slack command) and declare an Incident Commander. Follow the Incident Response Runbook (document 14) for the formal incident management process.

Common troubleshooting patterns:

- High latency —** check the service's latency breakdown in the Datadog APM trace view. Is the latency in the application code, the database, or an external dependency? If database, check for slow queries in the PostgreSQL slow_query_log. If external dependency, check if the dependency's status page shows an incident.
- High error rate —** check the error logs for the service (Datadog Logs, filtered by service name and level:error). Look for patterns: is the error from a specific endpoint? Is it affecting all users or a subset? Does it correlate with a recent deployment? If a recent deployment is suspect, consider rolling back using the ArgoCD rollback command.
- Pod restarts —** check the pod events (kubectl describe pod [name]) for OOMKilled (memory pressure) or CrashLoopBackOff (application crash). If OOMKilled, the pod may need a higher memory limit or may have a memory leak. If CrashLoopBackOff, check the pod logs for the crash reason.